



Frontend Developer — What Each Phase Contains

Phase 1 — Project Setup & UI Foundation

Priority:  Completed / Initial Setup

- React + TypeScript setup
 - Vite configuration
 - Tailwind CSS
 - shadcn/ui
 - React Router
 - React Hook Form
 - Zod
 - Framer Motion
 - Recharts
 - Project folder architecture
 - Common layouts
 - Responsive design foundation
 - API service structure
 - Environment configuration
 - Git/GitHub workflow
-

Phase 2 — Authentication & Authorization UI

Priority:  Current

Connect with Backend Phase 2.

Build:

- User registration
- User login
- Logout
- Forgot password
- Reset password
- Password validation
- Authentication state
- JWT token handling
- Protected routes
- User portal routing
- Admin login
- Photographer login

- Role-based navigation
 - Authentication error handling
 - Loading states
 - Form validation using Zod
-

Phase 3 — Common UI Components & Utilities

Priority: High

Create reusable components:

- Buttons
- Inputs
- Selects
- Modals
- Dialogs
- Dropdowns
- Cards
- Tables
- Pagination
- Search
- Filters
- Toast notifications
- Loading states
- Empty states
- Error states
- Confirmation dialogs
- Responsive sidebar
- Navbar
- Dashboard layout

Also establish reusable API/error-handling patterns.

Phase 4 — Membership Plans & Plan Selection

Priority: High

Build:

- Membership plans page
- Plan comparison
- Plan features
- Guest limit display
- Invitation-card/template limits
- Plan selection

- Selected-plan confirmation
- Current subscription display
- Plan restriction messages
- Upgrade/downgrade UI where supported
- Membership status

The UI must dynamically respect the limits returned by the backend.

Phase 5 — Event Management

Priority: High

Build the main event-management experience.

Users should be able to:

- Create event
- Edit event
- View event
- Delete event
- Event details
- Event name
- Event type
- Date
- Time
- Venue
- Location
- Google Maps/location link where supported
- Event description
- Event image
- Multiple event-related details
- Event status

The system must remain **generic for all types of events**, not marriage-only.

Examples:

- Wedding
 - Birthday
 - Engagement
 - Anniversary
 - Corporate event
 - Party
 - Conference
 - Other celebrations
-

Phase 6 — Guest Management

Priority: High

Build the guest-management system.

Features:

- Request contacts permission
- Import/select contacts
- Display contacts
- Add guest manually
- Edit guest
- Delete guest
- Guest name
- Phone number
- Email where applicable
- Guest group/household
- Guest count
- Search
- Filtering
- Pagination
- Guest status
- Invitation status

Guest table should provide a clean interface for managing the invitation list.

Phase 7 — Invitation Templates & Card Builder

Priority: High

Build:

- Invitation template gallery
- Template preview
- Template selection
- Plan-based template availability
- Invitation customization
- Event information preview
- Names/details
- Date
- Time
- Venue
- Location
- Custom invitation text
- Invitation card preview
- Mobile preview

- Desktop preview
- Save invitation
- Edit invitation
- Final invitation preview

The user should be able to clearly see what the guest will receive before sending.

Phase 8 — WhatsApp Invitation UI & Sending Flow

Priority: High

Build the invitation sending workflow.

User selects:

Guest
↓
Invitation
↓
Preview
↓
Send

Features:

- Select guests
- Select invitation
- Preview message
- Invitation link
- Invitation card
- Event details
- WhatsApp sending flow
- Sent status
- Failed status
- Pending status
- Resend option
- Individual sending
- Bulk sending where supported
- Sending confirmation

The invitation should contain a link that takes the guest to the response page.

Phase 9 — Guest Invitation Response UI

Priority: High

This replaces the traditional RSVP concept.

Guest receives:

Invitation



Invitation Link



Guest Response Page



Accept	Maybe	Reject	
--------	-------	--------	--

Build a public guest-facing page:

- Invitation details
- Event information
- Invitation card
- Guest name
- Accept button
- Maybe button
- Reject button
- Response confirmation
- Response update
- Mobile-friendly design

After the guest responds, the frontend should show the appropriate confirmation.

Phase 10 — User Dashboard & Analytics

Priority: Medium

Build the main user dashboard.

Display:

- Total events
- Active event
- Total guests
- Accepted guests
- Rejected guests
- Maybe guests
- Pending/unresponded guests
- Expected guest count
- Household/member count
- Invitations sent

- Invitations delivered where available
- Response statistics
- Gallery/media statistics
- Membership information

Use **Recharts** for useful visual analytics.

Phase 11 — Photographer Portal

Priority: Medium

Build a dedicated photographer/team interface.

Photographer should be able to:

- Login
- View assigned event
- Access event
- Upload photos
- Upload videos
- Upload multiple files
- Upload progress
- View uploaded files
- Delete/manage uploads where permitted
- See upload status
- Organize media

The photographer should have access **only to the event/media they are authorized to manage**.

Phase 12 — Gallery & Media Management

Priority: Medium

User portal:

- View photos
- View videos
- Photo grid
- Full-screen preview
- Video player
- Download
- Delete where authorized
- Media filtering
- Media organization

- Upload status
- Loading states
- Empty gallery state

Guest-facing gallery functionality should also be considered according to the backend's access rules.

Phase 13 — QR Code & Guest Photo Access

Priority: Medium

Build the QR-code workflow.

User portal:

- Display event QR code
- QR code preview
- Download QR code
- Download printable QR/PDF
- Print-friendly page

Guest flow:

Scan QR
↓
Open Guest Photo Access
↓
Camera / Photo Identification Flow
↓
Find Matching Photos
↓
Display Photos
↓
Download

The frontend should handle:

- QR landing page
 - Camera permission
 - Camera UI
 - Photo/selfie capture if required by the backend implementation
 - Upload/processing state
 - Matching result
 - Photo gallery
 - Download
-

Phase 14 — Admin Portal

Priority: High

Build a separate admin interface.

Admin should be able to view/manage:

- Users
- Events
- Memberships
- Guests
- Invitations
- Photographers
- Gallery/media
- QR codes
- System activity
- Dashboard statistics

Include:

- Search
 - Filters
 - Tables
 - Pagination
 - Detail pages
 - Edit actions
 - Delete/disable actions where permitted
 - Confirmation dialogs
 - Admin-only routes
-

Phase 15 — Notifications UI

Priority: Medium

Build:

- Notification dropdown
- Notification list
- Read/unread state
- Mark as read
- Clear/read-all
- Notification badge
- Relevant notification messages
- Error/success notifications

Examples:

- Invitation sent
 - Guest responded
 - Photographer uploaded media
 - Membership update
 - Event update
-

Phase 16 — Activity Logs & Audit UI

Priority: Medium

Build admin/user activity views where applicable.

Display:

- Activity
- User
- Action
- Event
- Date/time
- Status
- Relevant details

Example:

Mohamed
Created Event
Wedding Celebration
24 Aug 2026

Use clean tables and filtering.

Phase 17 — Performance, Security & UX Optimization

Priority: High

Before final testing:

- Route protection
- Role-based UI protection
- JWT handling
- API error handling
- Loading optimization
- Lazy loading
- Image optimization
- Pagination

- Form validation
 - Responsive testing
 - Mobile testing
 - Accessibility
 - Error boundaries
 - Network error handling
 - Empty states
 - Skeleton loaders
 - Confirmation dialogs
 - Prevent duplicate submissions
 - Secure frontend environment configuration
-

Phase 18 — Testing, Integration & Deployment

Priority: Final

Complete integration with the Django backend.

Test:

- Registration
- Login
- Logout
- Password reset
- Membership selection
- Event creation
- Guest management
- Invitation creation
- Invitation sending
- Guest Accept/Reject/Maybe
- Dashboard
- Photographer login
- Media upload
- Gallery
- QR flow
- Admin portal
- Notifications
- Permissions
- Responsive UI
- API failures
- Expired JWT
- Invalid data
- Empty data
- Production build

Recommended Parallel Development Flow

The two developers can now work like this:

